ChatGPT

Executive Summary

This report presents **five complete Colab notebooks** that together implement an end-to-end pipeline for "AI-Based Water Security Prediction for Africa". The notebooks cover dataset loading, cleaning, exploratory analysis, feature engineering, and machine learning, following best-practice data science workflows 【12†L137-L140】【10†L203-L206】. The code is organized so each Colab cell has a clear **markdown heading**, a brief explanation, and runnable Python code. We assume multiple CSV datasets (e.g. population, rainfall, water quality, infrastructure, vulnerability) in a cloned GitHub repo, which are automatically discovered and loaded. Unspecified details (e.g. GitHub token, target variable name, join keys) are noted as placeholders.

A schematic flow of the pipeline is shown below:

```
flowchart TD
    A[01_Data_Loading.ipynb<br/>(Dataset Loading)] -->
B[02_Data_Cleaning.ipynb<br/>(Cleaning)]
    B --> C[03_Exploratory_Data_Analysis.ipynb<br/>(EDA & Visualization)]
    C --> D[04_Feature_Engineering.ipynb<br/>(Merging & Feature Creation)]
    D --> E[05_Machine_Learning.ipynb<br/>(Modeling & Evaluation)]
    E --> F[Outputs<br/>(Figures, Models, Reports)]
```

The notebooks produce an **inventory table** of datasets and a **proposed variables table** for the merged modeling dataset, and save outputs under `Outputs/` (e.g. `Outputs/Figures/`, `Outputs/CleanData/`, `Outputs/Models/`). Throughout, Python libraries (pandas, numpy, scikit-learn, xgboost, shap, plotly) are used with clear inline comments. We cite authoritative sources for key methods and concepts.

Notebook 1: `01_Data_Loading.ipynb`

This notebook mounts Google Drive, clones the GitHub repo (using a personal access token), discovers CSV files, loads them into a dictionary, and creates a data inventory.

Cell 1: Install Required Packages

Install any needed Python packages.

```
# Colab typically has most libraries pre-installed, but we ensure up-to-date
versions
!pip install pandas numpy matplotlib plotly scikit-learn xgboost shap >/dev/null
```

Cell 2: Mount Google Drive

Mount the user's Google Drive to access files.

```
from google.colab import drive
drive.mount('/content/drive')  # will prompt for authentication【1†L1012-L1015】
```

Cell 3: Change Directory (optional)

Optionally change to a directory on Drive (e.g. a project folder).

```
# Change to a specific folder in Drive, or stay in root
%cd /content/drive/MyDrive/YourProjectFolder/  # replace with your Drive path
```

Cell 4: GitHub Authentication

Securely enter a GitHub token to clone the private repository.

```
from getpass import getpass

# Prompt for a GitHub personal access token (do NOT share your token publicly)
token = getpass("Enter your GitHub Personal Access Token: ")

# Define Git repo URL (replace <username> and <repo> with actual values)
github_user = "YourGitHubUsername"   # <-- placeholder
github_repo = "ANLOK-WATER-PREDICTION-PROJECT-"   # <-- placeholder (repo name)
!rm -rf $github_repo  # remove any existing folder
!git clone https://{token}@github.com/{github_user}/{github_repo}.git
```

> **Note:** Replace `YourGitHubUsername` and `ANLOK-WATER-PREDICTION-PROJECT-` with the actual GitHub user and repository name. Enter the token when prompted; it will not display on the screen【1†L1012-L1015】.

Cell 5: Verify Repository Files

List the cloned repository contents to confirm cloning.

```
import os
repo_path = f"/content/{github_repo}"
print("Repository structure:")
!ls "$repo_path"
```

Cell 6: Import Libraries

Import all necessary Python libraries.

```
# Data manipulation
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go

# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

# Statistics
from scipy.stats import zscore

# Utilities
import glob
import os
import warnings

warnings.filterwarnings("ignore")
plt.rcParams["figure.figsize"] = (10, 6)
```

Cell 7: Locate All CSV Files

Automatically find all `.csv` *files in the repository.*

```
# Use glob to recursively find CSV files (pandas, glob can locate files)【3†L84-L92】
csv_files = glob.glob(os.path.join(repo_path, "**/*.csv"), recursive=True)
print(f"Found {len(csv_files)} CSV files.")
for file in csv_files:
    print(file)
```

Cell 8: Load Datasets into Dictionary

Load each CSV into a pandas DataFrame stored in a dictionary.

```
datasets = {}
for file in csv_files:
    name = os.path.splitext(os.path.basename(file))[0]
    try:
        datasets[name] = pd.read_csv(file)
    except Exception as e:
        print(f"Skipping {name}: {e}")
        continue
print("Datasets loaded:", list(datasets.keys()))
```

Each dataset is now in `datasets[name]`, keyed by the filename (without `.csv`).

Cell 9: Create Inventory Table

Compute inventory (rows, columns, missing, duplicates, etc.) and display as DataFrame.

```
inventory = []
for name, df in datasets.items():
    inventory.append({
        "Dataset": name,
        "Rows": df.shape[0],
        "Columns": df.shape[1],
        "Memory(MB)": round(df.memory_usage().sum() / 1024**2, 2),
        "Missing Values": df.isnull().sum().sum(),
        "Duplicate Rows": df.duplicated().sum(),
        "Numeric Columns": len(df.select_dtypes(include=np.number).columns),
        "Categorical Columns": len(df.select_dtypes(exclude=np.number).columns)
    })

inventory_df = pd.DataFrame(inventory)
display(inventory_df)
```

Cell 10: Save Inventory Table

Save the inventory table to CSV for record.

```
outputs_dir = "Outputs"
os.makedirs(outputs_dir, exist_ok=True)
inventory_df.to_csv(os.path.join(outputs_dir, "dataset_inventory.csv"),
index=False)
print("Inventory saved to Outputs/dataset_inventory.csv")
```

Cell 11: Preview One Dataset

Example: show first rows of one dataset to verify loading.

```
# Replace with a valid dataset key if needed; here we try the first one
sample_key = list(datasets.keys())[0]
print(f"Sample dataset ({sample_key}):")
display(datasets[sample_key].head())
```

Inventory Table (example)
Below is the structure of the inventory table created above (values are examples):

Dataset	Rows	Columns	Memory(MB)	Missing Values	Duplicate Rows	Numeric Columns	Categorical Columns
Country_Demographics	1000	10	2.5	0	5	8	2
Monthly_Rainfall	1200	5	1.8	20	0	4	1
Water_Infrastructure	800	7	1.2	10	2	5	2
Water_Quality_Measures	950	6	1.4	0	0	6	0
Tariff_Economics	600	4	0.8	50	0	4	0
Population_Growth	1000	3	0.6	0	0	3	0
Vulnerability_Indices	900	5	1.0	15	1	4	1

(Note: The above is a placeholder example. The code will generate the actual inventory.)

Proposed Modeling Variables Table
We also outline a table of potential merged variables and their source datasets:

Variable Name	Source Dataset
Province	Country_Demographics
Country	Country_Demographics
Population	Country_Demographics
Population_Growth	Population_Growth
Urbanization_Rate	Country_Demographics
Annual_Rainfall	Monthly_Rainfall
Rainfall_Anomaly	Monthly_Rainfall
Water_Demand	Tariff_Economics

Variable Name	Source Dataset
Water_Tariff	Tariff_Economics
Infrastructure_Score	Water_Infrastructure
Quality_Score	Water_Quality_Measures
Vulnerability_Index	Vulnerability_Indices

(Note: Variable names and sources are illustrative. Actual merge keys are unspecified; use matching country/province columns.)

> **Usage:** In Colab, **run cells sequentially** from top to bottom to ensure all variables are defined before use (a common cause of `NameError` if skipped)【1†L1012-L1015】.

Notebook 2: `02_Data_Cleaning.ipynb`

This notebook imports the raw datasets, defines a cleaning function, removes duplicates, handles missing values, converts data types, detects outliers, and saves cleaned data.

Cell 1: Import Libraries

Load libraries and utilities for cleaning.

```
import pandas as pd
import numpy as np
from scipy.stats import zscore
import os, glob, warnings

warnings.filterwarnings("ignore")
```

Cell 2: Load Raw Datasets

Re-load the datasets (or import from previous step).

```
# Assuming we have a copy of the CSVs in the repo or from Outputs/CleanData
csv_files = glob.glob("/content/ANLOK-WATER-PREDICTION-PROJECT-/**/*.csv",
recursive=True)
datasets = {}
for file in csv_files:
    name = os.path.splitext(os.path.basename(file))[0]
    try:
        datasets[name] = pd.read_csv(file)
    except Exception as e:
```

```
        continue
print("Datasets to clean:", list(datasets.keys()))
```

Ensure the path points to the cloned repository. Variables from Notebook 1 (like `datasets`) are not automatically carried over, so we reload.

Cell 3: Define Cleaning Function

A reusable function to standardize DataFrames (drop duplicates, trim whitespace, etc.).

```
def clean_dataframe(df):
    df = df.copy()
    # Drop exact duplicate rows
    before = df.shape[0]
    df = df.drop_duplicates()
    after = df.shape[0]
    print(f"Dropped {before-after} duplicates")

    # Standardize column names: strip, lower-case, replace spaces with
underscores
    df.columns = df.columns.str.strip().str.lower().str.replace(r'\s+', '_',
regex=True)

    # Drop completely empty rows/columns
    df = df.dropna(axis=0, how='all').dropna(axis=1, how='all')

    # Strip whitespace from string entries
    for col in df.select_dtypes(include="object").columns:
        df[col] = df[col].astype(str).str.strip()
    return df
```

This function implements best practices for cleaning dataframes. It drops duplicate rows and trims text fields.

Cell 4: Apply Cleaning and Remove Duplicates

Clean each dataset using `clean_dataframe`, storing results.

```
cleaned = {}
for name, df in datasets.items():
    print(f"\nCleaning {name}:")
    df_clean = clean_dataframe(df)
    cleaned[name] = df_clean
```

Cell 5: Handle Data Types and Missing Values

Convert columns to appropriate dtypes; impute or flag missing values.

```
for name, df in cleaned.items():
    # Example: Convert date columns to datetime if any
    for col in df.columns:
        if 'date' in col or 'year' in col:
            try:
                df[col] = pd.to_datetime(df[col])
                print(f"Converted {name}.{col} to datetime")
            except:
                pass
    # Impute or flag missing numerical values
    numeric_cols = df.select_dtypes(include=np.number).columns
    if len(numeric_cols) > 0:
        imputer = SimpleImputer(strategy='mean')
        df[numeric_cols] = imputer.fit_transform(df[numeric_cols])
        print(f"Imputed missing numeric values in {name}")
    # For categorical: fill missing with 'Unknown'
    cat_cols = df.select_dtypes(include='object').columns
    for col in cat_cols:
        df[col] = df[col].fillna('Unknown')
    cleaned[name] = df
```

Numeric missing values are replaced by column means (`SimpleImputer(strategy='mean')`)【21†L645-L653】; categorical NAs are filled with `"Unknown"`.

Cell 6: Detect and Handle Outliers

Identify outliers using Z-scores; optionally cap or remove them.

```
for name, df in cleaned.items():
    numeric = df.select_dtypes(include=np.number)
    if not numeric.empty:
        z_scores = np.abs(zscore(numeric, nan_policy='omit'))
        outliers = (z_scores > 3).sum(axis=0)
        print(f"{name} outliers per column:\n{outliers}")
        # (Optional) Remove rows with any outlier in numeric columns:
        # df = df[(z_scores < 3).all(axis=1)]
        # cleaned[name] = df
```

This reports the count of values beyond 3 standard deviations. We could remove or cap them if needed (code commented out).

Cell 7: Create Cleaning Report

Summarize the effect of cleaning (rows, missing, duplicates) for each dataset.

```
report = []
for name, df in cleaned.items():
    report.append({
        "Dataset": name,
        "Rows": len(df),
        "Columns": df.shape[1],
        "Missing": df.isnull().sum().sum(),
        "Duplicates": df.duplicated().sum()
    })
report_df = pd.DataFrame(report)
display(report_df)
```

Cell 8: Save Cleaned Datasets

Write cleaned datasets to new CSV files under `Outputs/CleanData/`.

```
clean_dir = os.path.join("Outputs", "CleanData")
os.makedirs(clean_dir, exist_ok=True)
for name, df in cleaned.items():
    out_path = os.path.join(clean_dir, f"{name}_clean.csv")
    df.to_csv(out_path, index=False)
    print(f"Saved cleaned {name} ({df.shape}) to {out_path}")
```

> Each cleaned CSV is saved for future use (e.g. merged dataset in Notebook 4).

Notebook 3: `03_Exploratory_Data_Analysis.ipynb`

This notebook loads the cleaned datasets and performs EDA with visualizations. It includes heatmaps of missing values, numeric summaries, distribution plots, boxplots, correlation heatmaps, frequency tables for categoricals, and time-series plots if applicable. It saves figures under `Outputs/Figures/`.

Cell 1: Import Libraries and Load Clean Data

Load libraries for analysis and read the cleaned CSVs from `Outputs/CleanData`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
import os, glob

sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (10, 6)

clean_files = glob.glob("Outputs/CleanData/*_clean.csv")
data = {os.path.splitext(os.path.basename(f))[0].replace('_clean',''):
pd.read_csv(f)
        for f in clean_files}
print("Loaded cleaned datasets:", list(data.keys()))
```

Cell 2: Missing Value Heatmaps

Visualize missing values (if any) as a heatmap per dataset.

```
for name, df in data.items():
    plt.figure(figsize=(8, 4))
    sns.heatmap(df.isnull(), cbar=False)
    plt.title(f"Missing Values: {name}")
    plt.tight_layout()
    plt.savefig(f"Outputs/Figures/{name}_missing.png")
    plt.show()
```

This highlights where any missing data remain. Most should have been imputed.

Cell 3: Numeric Summaries

Show descriptive statistics for numeric columns.

```
for name, df in data.items():
    print(f"=== {name} Numeric Summary ===")
    display(df.describe().T.style.format("{:,.2f}"))
```

Provides count, mean, std, min/max for all numeric fields.

Cell 4: Distribution Plots (Histograms)

Plot histograms for each numeric variable to see distributions.

```
for name, df in data.items():
    numeric = df.select_dtypes(include=np.number)
```

```
    for col in numeric.columns:
        plt.figure()
        numeric[col].plot.hist(bins=30, edgecolor='k')
        plt.title(f"{name} - {col} Distribution")
        plt.xlabel(col)
        plt.ylabel("Frequency")
        plt.savefig(f"Outputs/Figures/{name}_{col}_hist.png")
        plt.show()
```

Cell 5: Boxplots for Outliers

Plot boxplots of numeric columns to identify outliers.

```
for name, df in data.items():
    numeric = df.select_dtypes(include=np.number)
    if not numeric.empty:
        numeric.plot(kind='box', vert=False)
        plt.title(f"{name} - Numeric Boxplots")
        plt.savefig(f"Outputs/Figures/{name}_boxplot.png")
        plt.show()
```

Cell 6: Correlation Heatmaps

Compute and visualize correlation matrices for numeric features.

```
for name, df in data.items():
    numeric = df.select_dtypes(include=np.number)
    if numeric.shape[1] > 1:
        corr = numeric.corr()
        plt.figure(figsize=(6, 5))
        sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm")
        plt.title(f"{name} - Correlation Matrix")
        plt.tight_layout()
        plt.savefig(f"Outputs/Figures/{name}_correlation.png")
        plt.show()
```

Correlation heatmaps help spot multicollinearity or strong relationships.

Cell 7: Categorical Value Counts

Print frequency tables for categorical columns.

```
for name, df in data.items():
    cat_cols = df.select_dtypes(include='object').columns
```

```
    for col in cat_cols:
        print(f"{name}.{col} value counts:")
        print(df[col].value_counts(dropna=False).head(10))
        print()
```

This shows the most common categories. Useful for understanding the data distribution.

Cell 8: Time-Series Plots

If datasets have date or year columns, plot trends over time.

```
for name, df in data.items():
    if 'date' in name.lower() or 'year' in name.lower():
        date_cols = [col for col in df.columns if 'date' in col.lower() or
'year' in col.lower()]
        for col in date_cols:
            try:
                df[col] = pd.to_datetime(df[col])
                plt.figure()
                ts = df.groupby(df[col].dt.year).mean(numeric_only=True)
                ts.plot(marker='o')
                plt.title(f"{name} - Trend by Year")
                plt.xlabel("Year"); plt.ylabel("Value")
                plt.savefig(f"Outputs/Figures/{name}_trend.png")
                plt.show()
            except Exception as e:
                continue
```

Example: plot average values per year if date columns exist. Adjust as needed for actual temporal patterns.

Cell 9: Interactive Plotly Dashboards (Optional)

Example: an interactive scatterplot of Rainfall vs. Water Demand.

```
# Example: assumes data has columns like 'annual_rainfall' and 'water_demand'
if 'Monthly_Rainfall' in data and 'Tariff_Economics' in data:
    df_r = data['Monthly_Rainfall']
    df_t = data['Tariff_Economics']
    # Merge on common index or region if possible (placeholder)
    demo = df_r.merge(df_t, on='province', how='inner')
    fig = px.scatter(demo, x='annual_rainfall', y='water_demand',
                     size='population', color='province',
                     title="Rainfall vs Water Demand",
                     labels={'annual_rainfall': 'Annual Rainfall (mm)',
```

```
'water_demand': 'Water Demand (ML)'})
    fig.write_html("Outputs/Figures/rainfall_vs_demand.html")
    fig.show()
```

> **Note:** Replace merge keys (`'province'`) with actual common columns. This produces an interactive HTML plot.

Each figure is saved under `Outputs/Figures/`. For example, `Outputs/Figures/Country_Demographics_missing.png`, etc.

Notebook 4: `04_Feature_Engineering.ipynb`

This notebook merges relevant datasets and creates new features such as per-capita metrics, anomalies, and indices. The result is a single final modeling table saved as `final_dataset.csv`.

Cell 1: Import and Load Clean Data

Load cleaned CSVs and prepare for merging.

```
import pandas as pd
import numpy as np
import os, glob

clean_files = glob.glob("Outputs/CleanData/*_clean.csv", recursive=True)
data = {os.path.splitext(os.path.basename(f))[0].replace('_clean',''):
pd.read_csv(f)
        for f in clean_files}
print("Datasets loaded for merging:", list(data.keys()))
```

Cell 2: Define Merge Keys (Placeholder)

Specify the join keys and merge strategy.

```
# Example keys (modify based on actual data)
key_cols = ['country', 'province', 'year']  # common keys for merging
print("Join keys (columns):", key_cols)
```

> **Note:** Adjust `key_cols` to actual columns present in all datasets (e.g. `'CountryID'`, `'ProvinceID'`, `'Year'`). All merges here use an outer join for completeness.

Cell 3: Merge Datasets

Sequentially merge all datasets into one DataFrame.

```
# Start with demographics as the base
final_df = data.get('Country_Demographics', pd.DataFrame())
for name, df in data.items():
    if name == 'Country_Demographics':
        continue
    try:
        final_df = final_df.merge(df, on=key_cols, how='left',
suffixes=('','_'+name))
        print(f"Merged with {name}")
    except Exception as e:
        print(f"Could not merge {name}: {e}")
```

Adjust the merge order or keys as needed if some joins fail (the code will print any issues).

Cell 4: Create Population Features

Example: Population density, urbanization rate, population growth rate.

```
if 'Country_Demographics' in data:
    demo = data['Country_Demographics']
    # Assuming columns 'population' and 'area_sq_km'
    if {'population', 'area_sq_km'}.issubset(demo.columns):
        demo['pop_density'] = demo['population'] / demo['area_sq_km']
    if {'pop_growth_rate', 'population'}.issubset(demo.columns):
        demo['pop_change'] = demo['population'] * demo['pop_growth_rate'] / 100
    # Merge back to final_df
    final_df =
final_df.merge(demo[['country','province','year','pop_density','pop_change']],
                               on=key_cols, how='left')
```

Cell 5: Create Rainfall Features

Example: Rainfall anomaly (difference from regional mean).

```
if 'Monthly_Rainfall' in data:
    rain = data['Monthly_Rainfall']
    # Assume 'annual_rainfall' column exists
    if 'annual_rainfall' in rain.columns:
        rain['rainfall_anomaly'] = rain['annual_rainfall'] -
rain['annual_rainfall'].mean()
        final_df =
final_df.merge(rain[['country','province','year','annual_rainfall','rainfall_anomaly']],
                                   on=key_cols, how='left')
```

Cell 6: Infrastructure and Quality Features

Example: Infrastructure per 1000 people, water quality index.

```
if 'Water_Infrastructure' in data:
    infra = data['Water_Infrastructure']
    # Assume 'infrastructure_count' and 'population'
    if {'infrastructure_count', 'population'}.issubset(infra.columns):
        infra['infra_per_1000'] = infra['infrastructure_count'] /
(infra['population']/1000)
        final_df =
final_df.merge(infra[['country','province','year','infra_per_1000']],
                                   on=key_cols, how='left')

if 'Water_Quality_Measures' in data:
    quality = data['Water_Quality_Measures']
    # Suppose 'quality_score' already present or compute one
    if 'quality_score' not in quality.columns and 'contaminant_level' in
quality.columns:
        quality['quality_score'] = 100 - quality['contaminant_level']  #
simplistic example
    final_df =
final_df.merge(quality[['country','province','year','quality_score']],
                               on=key_cols, how='left')
```

Cell 7: Demand and Tariff Features

Example: Water demand growth, tariff per household.

```
if 'Tariff_Economics' in data:
    econ = data['Tariff_Economics']
    if {'water_demand', 'water_demand_lag'}.issubset(econ.columns):
        econ['demand_growth'] = (econ['water_demand'] -
econ['water_demand_lag']) / econ['water_demand_lag']
    if {'tariff_per_cubic_m', 'average_household_size'}.issubset(econ.columns):
        econ['tariff_per_household'] = econ['tariff_per_cubic_m'] *
econ['average_household_size']
    final_df =
final_df.merge(econ[['country','province','year','water_demand','demand_growth','tariff_per_house
                               on=key_cols, how='left')
```

Cell 8: Feature Selection (Dropping Redundant Columns)

Drop any columns not needed for modeling.

```
# Example: drop raw counts if per-capita metrics exist
drop_cols = ['population', 'infrastructure_count', 'water_demand_lag',
'contaminant_level']
final_df = final_df.drop(columns=[c for c in drop_cols if c in
final_df.columns], errors='ignore')
print("Final features:", list(final_df.columns))
```

Cell 9: Save Final Dataset

Write the merged and feature-engineered table for modeling.

```
final_df.to_csv("Outputs/Final_Modeling_Table.csv", index=False)
print(f"Final dataset saved with shape {final_df.shape}")
```

Notebook 5: `05_Machine_Learning.ipynb`

This notebook loads the final dataset, splits into train/test sets, scales features, trains multiple models (Linear Regression, Random Forest, XGBoost), tunes hyperparameters, evaluates performance (MAE, RMSE, R^2) and uses SHAP for explainability. The best model is saved to `Outputs/Models/`.

Cell 1: Import Libraries and Load Data

Load necessary ML libraries and the final dataset.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
import xgboost as xgb
import shap
import joblib

df = pd.read_csv("Outputs/Final_Modeling_Table.csv")
print("Data shape:", df.shape)
```

Cell 2: Define Features and Target

Specify the target variable and feature columns.

```
# Placeholder target column (to be replaced with actual name)
target = "water_security_index"  # <-- Replace with actual target column
if target not in df.columns:
    raise ValueError(f"Target column '{target}' not found in data.")

# Exclude key and target to get feature list
feature_cols = [c for c in df.columns if c not in [target,
'country','province','year']]
X = df[feature_cols]
y = df[target]
print(f"Features: {len(feature_cols)} | Target: {target}")
```

> **Note:** Replace `"water_security_index"` with the actual target variable name (e.g. a vulnerability score). Ensure `feature_cols` excludes non-predictive identifiers.

Cell 3: Train-Test Split

Split data into training and testing subsets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Train size:", X_train.shape, "Test size:", X_test.shape)
```

> This uses scikit-learn's `train_test_split` 【19†L645-L649】 for an 80/20 split.

Cell 4: Feature Scaling

Scale features using a StandardScaler (fit on training data only).

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

> Scaling often improves model training, especially for regularization and distance-based models.

Cell 5: Baseline Linear Regression

Train a simple Linear Regression as a baseline model.

```
lr = LinearRegression()
lr.fit(X_train_scaled, y_train)
```

```
y_pred_lr = lr.predict(X_test_scaled)
lr_mae = mean_absolute_error(y_test, y_pred_lr)
lr_rmse = mean_squared_error(y_test, y_pred_lr, squared=False)
lr_r2 = r2_score(y_test, y_pred_lr)
print(f"Linear Regression: MAE={lr_mae:.2f}, RMSE={lr_rmse:.2f}, R^2={lr_r2:.
2f}")
```

This provides a benchmark performance.

Cell 6: Random Forest Regressor (Grid Search)

Train a Random Forest with hyperparameter tuning (e.g. number of trees).

```
rf = RandomForestRegressor(random_state=42)
param_grid = {'n_estimators': [100, 300], 'max_depth': [None, 10, 20]}
grid_rf = GridSearchCV(rf, param_grid, cv=3, scoring='neg_mean_absolute_error')
grid_rf.fit(X_train_scaled, y_train)
best_rf = grid_rf.best_estimator_
y_pred_rf = best_rf.predict(X_test_scaled)
rf_mae = mean_absolute_error(y_test, y_pred_rf)
rf_rmse = mean_squared_error(y_test, y_pred_rf, squared=False)
rf_r2 = r2_score(y_test, y_pred_rf)
print(f"Random Forest (best): {grid_rf.best_params_} -> MAE={rf_mae:.2f},
RMSE={rf_rmse:.2f}, R^2={rf_r2:.2f}")
```

Random Forest often captures nonlinear relationships. We tune parameters with `GridSearchCV`.

Cell 7: XGBoost Regressor

Train an XGBoost model (fast tree-based model).

```
xgbr = xgb.XGBRegressor(random_state=42)
param_grid = {'n_estimators': [100, 200], 'learning_rate': [0.05, 0.1],
'max_depth': [3, 6]}
grid_xgb = GridSearchCV(xgbr, param_grid, cv=3,
scoring='neg_mean_absolute_error')
grid_xgb.fit(X_train_scaled, y_train)
best_xgb = grid_xgb.best_estimator_
y_pred_xgb = best_xgb.predict(X_test_scaled)
xgb_mae = mean_absolute_error(y_test, y_pred_xgb)
xgb_rmse = mean_squared_error(y_test, y_pred_xgb, squared=False)
xgb_r2 = r2_score(y_test, y_pred_xgb)
```

```
print(f"XGBoost (best): {grid_xgb.best_params_} -> MAE={xgb_mae:.2f},
RMSE={xgb_rmse:.2f}, R^2={xgb_r2:.2f}")
```

XGBoost is widely used for structured data due to its performance. Here we tune basic parameters.

Cell 8: Model Comparison Table

Compare models using MAE, RMSE, R^2.

```
results = pd.DataFrame({
    "Model": ["LinearRegression", "RandomForest", "XGBoost"],
    "MAE": [lr_mae, rf_mae, xgb_mae],
    "RMSE": [lr_rmse, rf_rmse, xgb_rmse],
    "R2": [lr_r2, rf_r2, xgb_r2]
})
display(results)
results.to_csv("Outputs/model_performance_comparison.csv", index=False)
```

This table helps choose the best model. Usually the one with highest R^2 and lowest error.

Cell 9: Feature Importance (SHAP)

Use SHAP to explain the best model's predictions.

```
# Assume best_xgb is best; use SHAP TreeExplainer (supports XGB, RF)
explainer = shap.Explainer(best_xgb)
shap_values = explainer(X_test_scaled)
# Summary plot (global feature importance)
shap.summary_plot(shap_values, X_test, feature_names=feature_cols, show=False)
plt.savefig("Outputs/Figures/shap_summary.png")
```

SHAP values explain each feature's contribution【10†L203-L206】. A summary plot (saved above) shows feature impact on predictions.

Cell 10: Save Best Model

Serialize the best model to a file for future inference.

```
os.makedirs("Outputs/Models", exist_ok=True)
joblib.dump(best_xgb, "Outputs/Models/best_model_xgb.pkl")
print("Best model (XGBoost) saved to Outputs/Models/best_model_xgb.pkl")
```

Cell 11: Prediction Dashboard (Optional)

Example: Predict and display a few sample results.

```
sample_idx = np.random.choice(len(X_test), 5, replace=False)
for idx in sample_idx:
    features = X_test.iloc[idx:idx+1]
    pred = best_xgb.predict(scaler.transform(features))[0]
    actual = y_test.iloc[idx]
    print(f"Sample {idx}: Actual={actual:.2f}, Predicted={pred:.2f}")
```

Troubleshooting: If a `NameError` occurs (e.g. `df` not defined), ensure that all previous cells were run in order. Colab requires running from top to bottom so that variables (like `df`, `X_train`, etc.) exist. Use "Runtime → Run all" to execute all cells sequentially if needed.

References

- Pandas documentation for `drop_duplicates`, `read_csv`, `merge`, etc.【6†L248-L257】【15†L39-L47】.
- Scikit-learn documentation for `train_test_split` and `SimpleImputer`【19†L645-L649】【21†L645-L653】.
- SHAP documentation for explainability in ML models【10†L203-L206】.
- Examples of ML for water forecasting and urban planning【12†L137-L140】【10†L242-L250】.
- Colab-specific guides (Drive mounting and Git cloning)【1†L1012-L1015】【1†L1023-L1026】.
